```
/*
Expected Output:
{
  res2: {
    text: "You just asked for an example of artificial intelligence."
  }
}
*/
```

Once you create an instance of the `ConversationChain` component, you can integrate it into other chains and perform more complex queries. This highlights the power of functional composability that the LangChain project provides.

Regarding the first step of retrieving past messages from a database, LangChain offers reference integrations for popular databases, enabling you to store and retrieve messages in a format that is human readable and easily indexable for searching. You can find the official list of supported integrations on the LangChain documentation page (https://js.langchain.com/docs/integrations/memory).

Files: memory.openai.js, memory.google.js

The code for the project, showing conversational memory in LangChain, is in the ch06 folder in the repository. You can use that example by running this command in the source folder if using OpenAI:

```
$ node memory.openai.js
```

If you are using Google AI, there is a separate file named memory.google.js in the same folder:

```
$ node memory.google.js
```

You'll need an OpenAI API key to access their REST API or a Google API key for Google services. For details on creating API keys, see the appendix:

```
// .env

OPENAI_API_KEY=<SECRET_KEY>
GEMINI_API_KEY=<YOUR_GEMINI_API_KEY>
```

When implementing conversation memory in LangChain.js, keep the following in mind:

- *Prompt configuration*—If you wish to customize the prompt of the chain, you need to override the `prompt` parameter in the `ConversationChain` constructor. Ensure that you include the relevant variables for `{history}` and `{input}` in your prompt to maintain context effectively; otherwise, your requests will fail to complete.